

第7章：遗传算法

张正



哈尔滨工业大学（深圳）

zhengzhang@hit.edu.cn

<http://faculty.hitsz.edu.cn/zhangzheng>

机器学习的流派之争

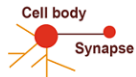
符号主义者



起源：逻辑学、哲学 | 核心思想：认知即计算，**符号的演绎和逆演绎**
问题：知识结构 | 代表算法：逆演绎算法（Inverse deduction）
应用：知识图谱 | 代表人物是：Tom Mitchell, Steve Muggleton, Ross Quinlan



联结主义者



起源：神经科学 | 核心思想：对大脑进行仿真
问题：信度分配 | 代表算法：**BP、深度学习**
应用：CV、SR | 代表人物是：Bengio, Hinton, LeCun



进化主义



起源：进化生物学 | 核心思想：**进化模拟，遗传算法和遗传编程**
问题：结构发现 | 代表算法：基因编程（Genetic programming）
应用：海星机器人 | 代表人物：John Koda、John Holland、Hod Lipson



贝叶斯派



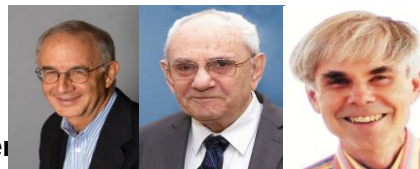
起源：统计学 | 核心思想：**概率估计，概率修正，最优决策**
问题：不确定性 | 代表算法：概率推理（Probabilistic inference）
应用：概率预测 | 代表人物：David Heckerman, Judea Pearl, Michael Jordan



行为类比主义



起源：心理学 | 核心思想：新旧知识间的相似性
问题：相似性 | 代表算法：**核机器、近邻算法**
应用：Netflix推荐系统 | 代表人物：Peter Hart, Vladimir Vapnik, Douglas Hofstadter



本章内容

一、演化计算基础知识

二、遗传算法

三、遗传算法举例

三、假设空间搜索

一、演化学习基础知识

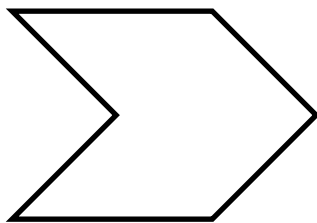
定义：演化学习基于演化算法提供的优化工具设计机器学习算法。

演化算法

- 遗传算法
- 演化规划
-

机器学习

- 神经网络
- 强化学习
-



演化学习

- 演化特征学习
- 演化监督学习
- 演化非监督学习
- 演化强化学习
-

一、演化学习基础知识

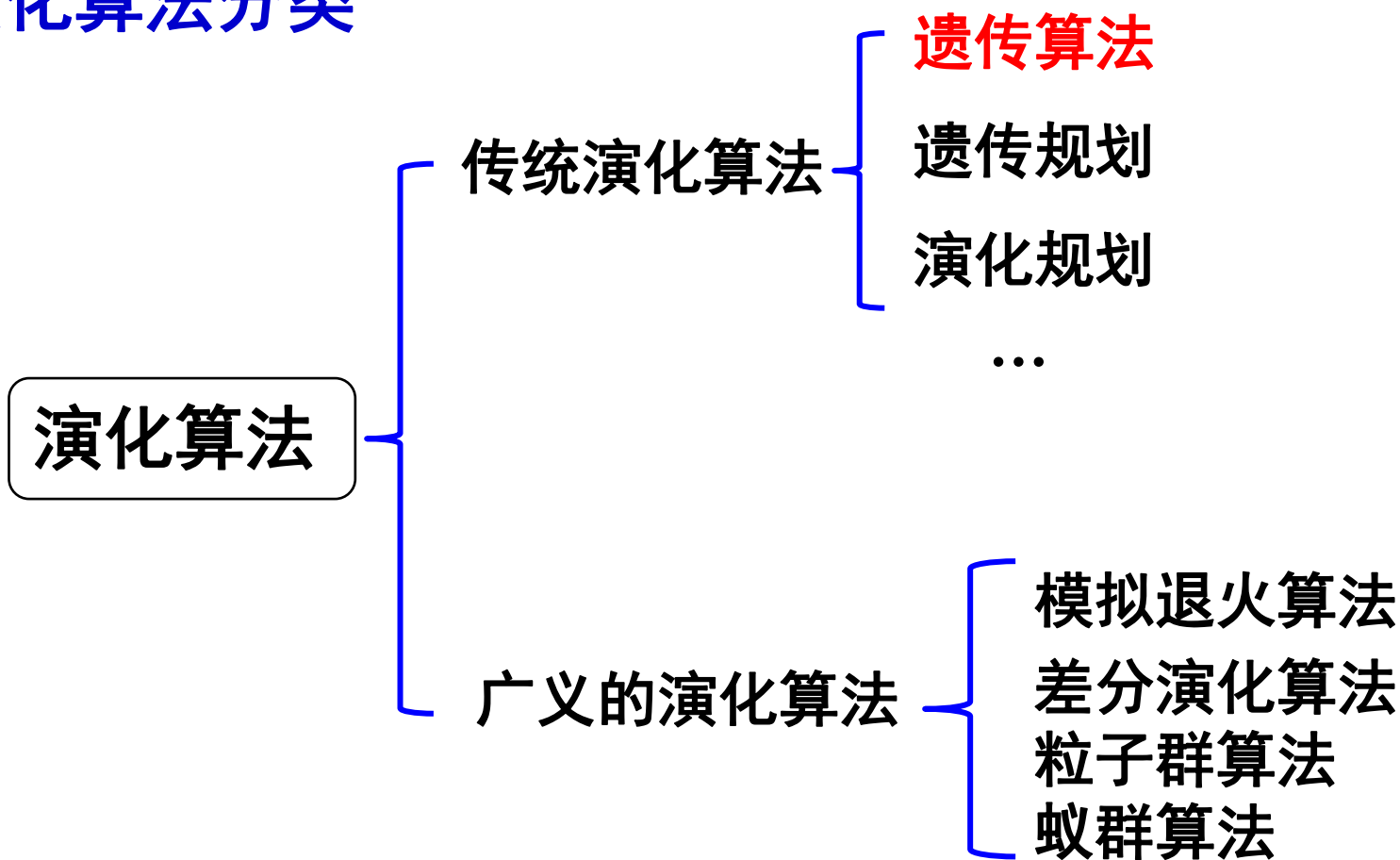
演化算法：或称“进化算法”，它是一个“算法簇”，其灵感都来自于大自然的生物进化。演化算法有很多版本，比如，有**不同的遗传基因表达方式**，不同的**交叉和变异算子**，以及**不同的再生和选择方法**。

与传统的优化算法相比，**演化算法的特点**在于：

- ◆ 具有高鲁棒性和广泛适应性；
- ◆ 具有自组织、自适应、自学习的特性；
- ◆ 本质并行性；
- ◆ 能够不受问题性质的限制，有效处理传统优化算法难以解决的复杂问题。

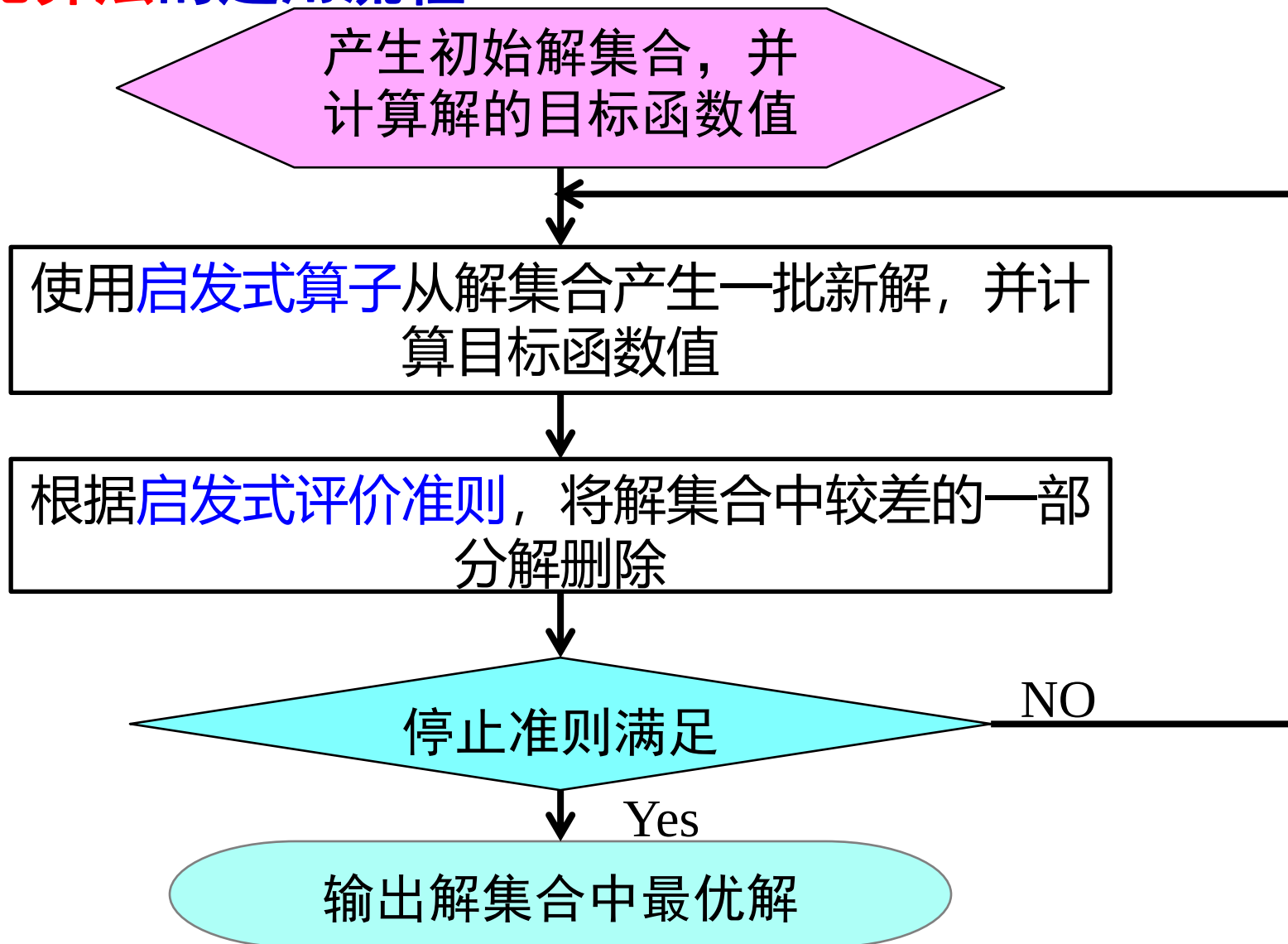
一、演化学习基础知识

演化算法分类



一、演化学习基础知识

演化算法的通用流程



二、遗传算法

定义：遗传算法是模拟生物在自然环境中的遗传和演化过程而形成的一种自适应全局优化概率搜索算法。

历史：

1. 遗传算法最早由美国密西根大学的 J. Holland 教授提出，起源于20世纪60年代对自然和人工自适应系统的研究。
2. 70年代，De Jong 基于遗传算法的思想在计算机上进行了大量的纯数值函数优化计算实验。
3. 80年代，在前人的基础上，由Goldberg进行归纳总结，形成了遗传算法的基本框架。

二、遗传算法

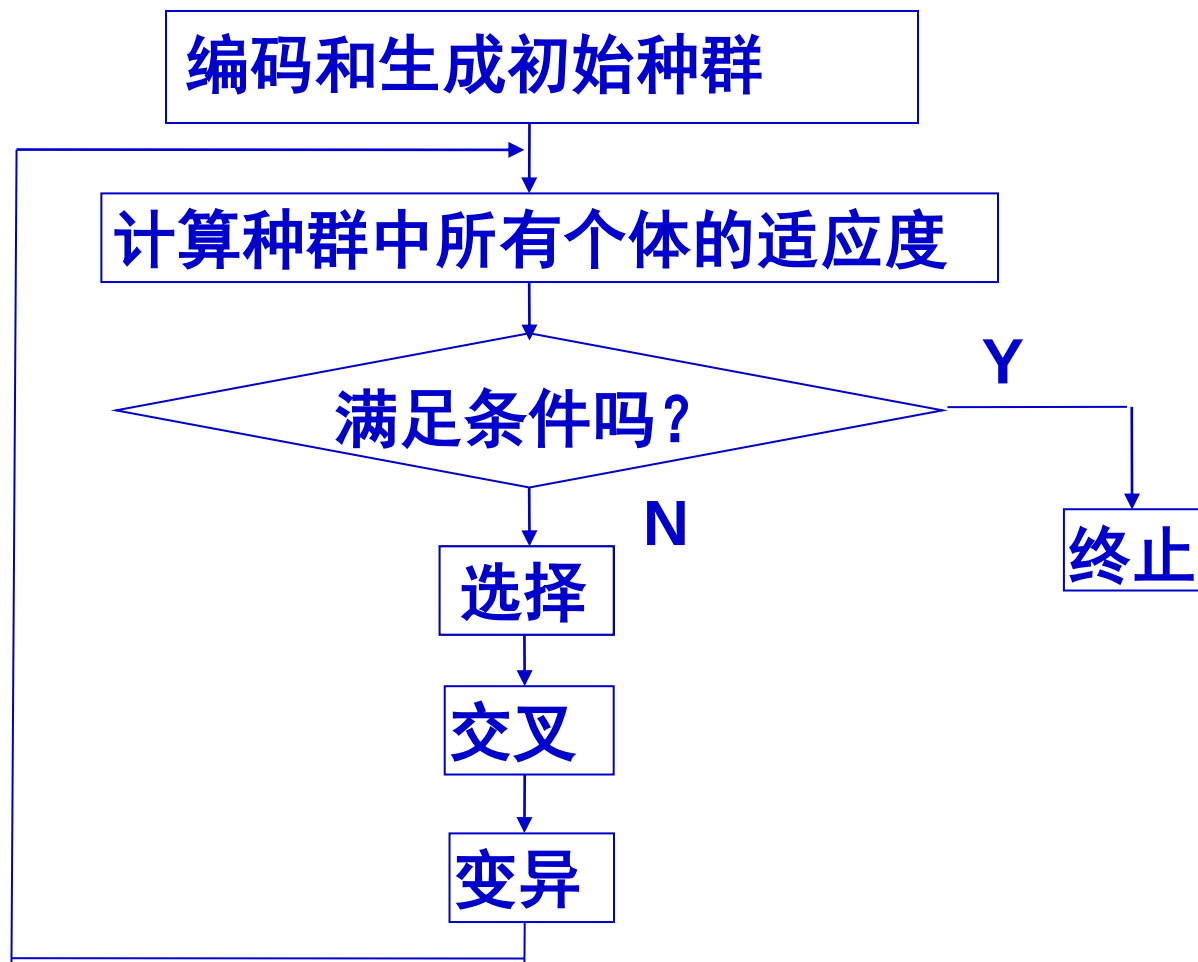
● 遗传算法的基本思想

从初始种群出发，采用优胜劣汰、适者生存的自然法则选择个体，并通过杂交、变异来产生新一代种群，如此逐代演化，直到满足目标为止。

● 遗传算法的特点

- 遗传算法是从问题解空间**多点并行搜索**，而非从单个解开始搜索（并行性）；
- 遗传算法利用目标函数的**适应度**这一信息而非利用导数或其它辅助信息来指导搜索（泛化性）；
- 遗传算法利用**选择、交叉、变异**等算子，而不是利用确定性规则进行操作（随机性）。

二、遗传算法



基本遗传算法的算法流程图

二、遗传算法

- 搜索合适的假设是从若干初始假设的群体或集合开始
 - 通过模仿生物进化产生下一代群体
 - 每一步根据给定的适应度评估当前群体中的假设
 - 随后，使用概率方法选出适应度最高的假设作为下一代群体的种子
- 遗传算法 (Genetic Algorithm, GA) 是一种受生物进化启发的学习方法
 - 通过不断变异和重组当前已知最好的假设生成后续假设
- 遗传算法的发展得益于如下因素
 - 进化是一种成功的自适应方法，具有较好的健壮性
 - 遗传算法搜索的假设空间中，假设的各个部分相互作用，每一部分对总的假设适应度的影响难以建模
 - 易于并行化
 - 遗传算法和遗传编程都是进化计算领域的重要方法

二、遗传算法

- 遗传算法研究的问题
 - 搜索候选假设空间并确定最佳假设
- 最佳假设
 - **适应度**：当前问题预先定义的数字**度量**
 - 学习任务是实现输入输出一个逼近，适应度就是精度
 - 最佳假设拥有最优的适应度
- 进化计算的共同结构
 - 迭代**更新一个假设池**，即群体
 - 根据**适应度函数**评估群体中的每一个个体
 - 用概率方法**选取适应度最高**的个体生成新一代群体

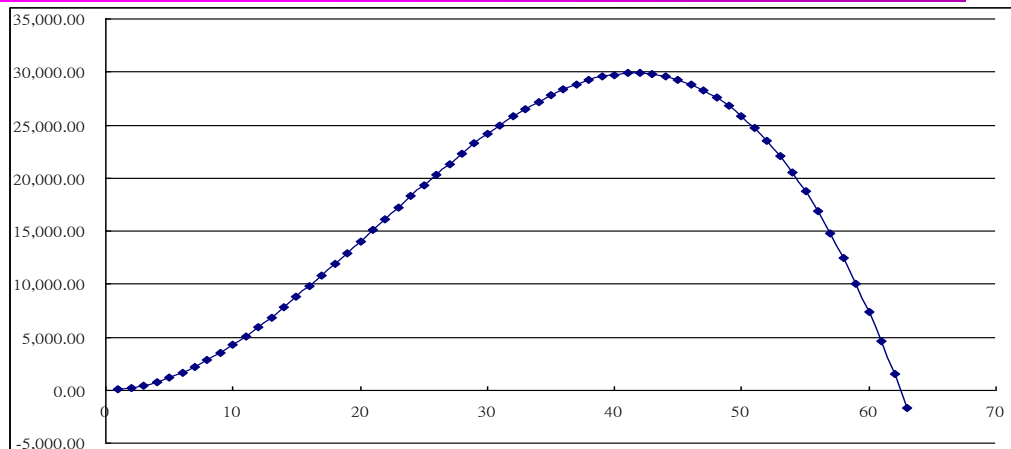
二、遗传算法

试解决如下问题：

$$\text{Max } F(X) = -0.83X^3 + 52X^2 - 8.5X + 8$$

$$\text{Subject to: } 0 \leq X \leq 61$$

$$X \in \text{Integer}$$



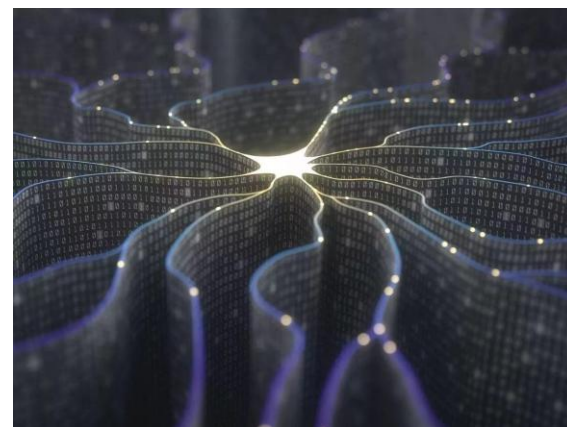
● 解决方案？

- 理论法，如果没有解？
- 直接法（牛顿，共轭梯度等）
- 蛮力法，如果搜索空间很大？
- 其他方法？

二、遗传算法

“A Dumb Solution”

- “盲目生成测试” 算法
 - 重复
 - 生成一个任意可能的解
 - 测试解的性能
 - 直到解够好
- 可行性如何呢？
 - 有时-可以
 - 如果只有些许可可能的解
 - 你有足够时间
 - 大多数时候-不可以
 - 可能的解空间太大
 - 没时间去一一尝试



二、遗传算法

一个“less-dumb”的方法

- 遗传算法：

- 生成一个任意解的集合
- 重复
 - 测试集合中每个解（将它们排序）
 - 从集合中移除一些不好的解
 - 复制一些好的解
 - 对它们做出一些小的改变
- 直到最好的解足够好

二、遗传算法

遗传算法编码

二进制编码

二进制编码是将原问题的结构变换为染色体的位串结构。在二进制编码中，首先要确定二进制字符串的长度，该长度与变量的定义域和所求问题的计算精度有关。

例如：假设变量 x 的定义域为 $[-2, 5]$ ，其要求精度为 $10E^{-6}$ ，则需要将 $[-2, 5]$ 分成7000000个等长小区间，每个小区间用一个二进制位串来表示，于是二进制位串长度至少23位。这是因为：

$$4194304 = 2^{22} < 7000000 < 2^{23} = 8388608$$

二进制编码存在的主要缺点：汉明悬崖。

例如，7和8的二进制数分别为0111和1000，当算法从7改进到8时，就必须改变所有的位。

二、遗传算法

练习一：对于编码长度为7的二进制编码，判断以下编码的合法性：

(1) [1 0 2 0 1 1 0]

(2) [1 0 1 1 1 0 1]

(3) [1 0 1 0 1 0 0]

(4) [0 0 0 0 0 0 0]

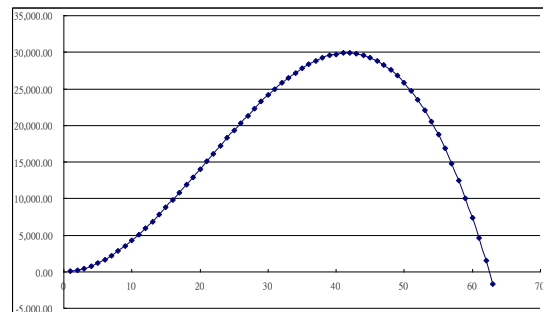
(5) [2 1 3 4 7 6 8]

练习二、在遗传算法中，变量x取0到61之间的整数值，如何对x进行二进制编码？

二、遗传算法

$$\text{Max } F(X) = -0.83X^3 + 52X^2 - 8.5X + 8$$

Subject to: $0 \leq X \leq 61, X \in \text{Integer}$



● Step 1: 二进制编码

X

F(X)

pop 1 **43**

1	0	1	0	1	1
---	---	---	---	---	---

29799.69

pop 2 **6**

0	0	0	1	1	0
---	---	---	---	---	---

1649.72

pop 3 **35**

1	0	0	0	1	1
---	---	---	---	---	---

27824.25

● Step 2: 生成群体

pop 1	0	1	1	1	0	1	29
pop 2	1	0	1	0	1	0	42
pop 3	0	1	1	1	1	1	31
pop 4	0	1	0	1	0	1	21
pop 5	1	0	0	1	1	1	39
...	.	.	.	.	.	.	.
...	.	.	.	.	.	.	.
pop 19	0	1	0	0	0	1	17
pop 20	0	0	1	1	1	1	15

二、遗传算法

● Step3: 选择(比例)

Survival Rate=
fitness/ sum(fitness)
适应度

	X							F(X)	Survival rate
Pop 1	0	1	1	1	0	1	29	23250.63	5.77%
Pop 2	1	0	1	0	1	0	42	29885.96	7.41%
Pop 3	0	1	1	1	1	1	31	24989.97	6.20%
Pop 4	0	1	0	1	0	1	21	15074.87	3.74%
Pop 5	1	0	0	1	1	1	39	29533.73	7.33%
Pop 6	1	0	1	1	1	1	47	28303.41	7.02%
Pop 7	0	1	0	1	0	1	21	15074.87	3.74%
Pop 8	0	1	0	0	0	1	17	10813.71	2.68%
Pop 9	0	0	1	1	1	1	15	8779.25	2.18%
Pop 10	1	1	0	1	1	1	55	18749.25	4.65%
Pop 11	0	1	1	1	0	1	29	23250.63	5.77%
Pop 12	0	1	0	1	1	1	23	17221.89	4.27%
Pop 13	1	0	1	0	1	1	43	29799.69	7.39%
Pop 14	0	1	1	0	1	0	26	20350.92	5.05%
Pop 15	1	0	0	0	1	1	35	27824.25	6.90%
Pop 16	0	0	1	1	1	1	15	8779.25	2.18%
Pop 17	0	1	0	1	1	1	23	17221.89	4.27%
Pop 18	0	0	0	1	0	1	5	1161.75	0.29%
Pop 19	1	0	1	0	1	1	43	29799.69	7.39%
Pop 20	0	1	1	1	0	1	29	23250.63	5.77%

二、遗传算法

● Step 4: 交叉

pop 1	0	1	1	1	0	1	29
pop 2	1	0	1	0	1	0	42
pop 3	0	1	1	1	1	1	31
pop 4	0	1	0	1	0	1	21
pop 5	1	0	0	1	1	1	39
...	.	.	.	.	.	.	.
...	.	.	.	.	.	.	.
pop 19	0	1	0	0	0	1	17
pop 20	0	0	1	1	1	1	15

随机选择
两个双亲

Parent 1

0	1	1	1	0	1
---	---	---	---	---	---

Parent 2

1	0	0	1	1	1
---	---	---	---	---	---

Crossover point

Offspring 1

0	1	1	1	1	1
---	---	---	---	---	---

X

F(X)

31

24989.97

Offspring 2

1	0	0	1	0	1
---	---	---	---	---	---

37

28839.51

二、遗传算法

● Step 5: 变异

		X	F(X)			X	F(X)												
Offspring 1	<table><tr><td>0</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td></tr></table>	0	1	1	1	1	1	31	24989.97										
0	1	1	1	1	1														
Offspring 2	<table><tr><td>1</td><td>0</td><td>0</td><td>1</td><td>0</td><td>1</td></tr></table>	1	0	0	1	0	1	37	28839.51	→	<table><tr><td>1</td><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td></tr></table>	1	0	0	1	1	1	39	29533.73
1	0	0	1	0	1														
1	0	0	1	1	1														
Offspring 3	<table><tr><td>0</td><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td></tr></table>	0	0	1	0	1	0	10	4293.00										
0	0	1	0	1	0														
.....																			
Offspring 20	<table><tr><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td></tr></table>	0	0	0	1	1	0	6	1649.72										
0	0	0	1	1	0														

↑

随机选取变异位点

● Step 6: 回到步骤3

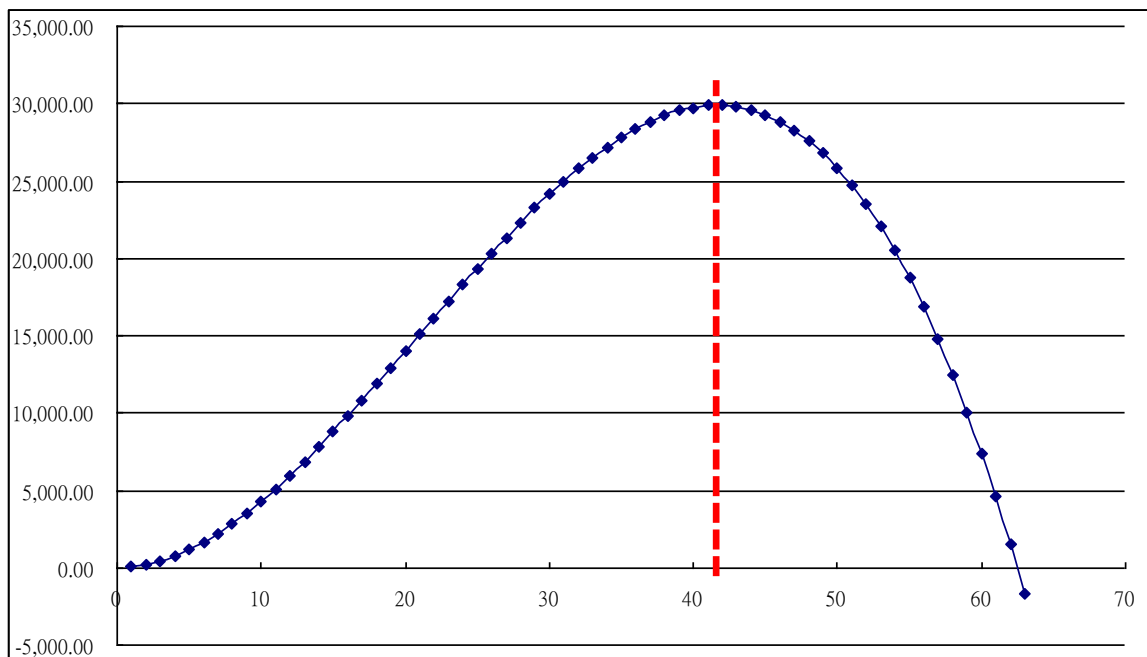
二、遗传算法

● 直到收敛

							X	F(X)
Pop 1	1	0	1	0	1	0	42	29885.96
Pop 2	1	0	1	0	1	0	42	29885.96
Pop 3	1	0	1	0	1	0	42	29885.96
Pop 4	1	0	1	0	1	0	42	29885.96
Pop 5	1	0	1	0	1	0	42	29885.96
Pop 6	1	0	1	0	1	0	42	29885.96
Pop 7	1	0	1	0	1	0	42	29885.96
Pop 8	1	0	1	0	1	0	42	29885.96
Pop 9	1	0	1	0	1	0	42	29885.96
Pop 10	1	0	1	0	1	0	42	29885.96
Pop 11	1	0	1	0	1	0	42	29885.96
Pop 12	1	0	1	0	1	0	42	29885.96
Pop 13	1	0	1	0	1	0	42	29885.96
Pop 14	1	0	1	0	1	0	42	29885.96
Pop 15	1	0	1	0	1	0	42	29885.96
Pop 16	1	0	1	0	1	0	42	29885.96
Pop 17	1	0	1	0	1	0	42	29885.96
Pop 18	1	0	1	0	1	0	42	29885.96
Pop 19	1	0	1	0	1	0	42	29885.96
Pop 20	1	0	1	0	1	0	42	29885.96

$$\text{Max } F(X) = -0.83X^3 + 52X^2 - 8.5X + 8$$

Subject to: $0 \leq X \leq 61, X \in \text{Integer}$



二、遗传算法原型

- 遗传算法(Fitness, fitness_threshold, p, r, m)
 - Fitness: 适应度评分函数
 - fitness_threshold: 指定终止判据的阈值
 - p: 群体中包含的假设数量
 - r: 每一步中通过交叉取代群体成员的比例
 - m: 变异率 (单点、多点)

二、遗传算法原型

- 初始化群体: $P \leftarrow$ 随机产生 p 个假设
- 评估: 对于 P 中每一个 h , 计算 $\text{Fitness}(h)$
- 当 $[\max \text{Fitness}(h)] < \text{Fitness_threshold}$ 时, 做:
 - { 产生 **新一代群体** P_S :
 1. **选择**: 用 **概率方法** 选择 P 的 $(1-r)p$ 个成员加入到 P_S , 从 P 中选择假设 h_i 的概率定义为:
$$\Pr(h_i) = \text{Fitness}(h_i) / \sum_j^p \text{Fitness}(h_j)$$
 2. **交叉**: 从 P 中按概率选择 $r \cdot p/2$ 对假设。对每对假设 $\langle h_1, h_2 \rangle$, 应用交叉算子产生两个后代; 把所有的后代加入到 P_S ;
 3. **变异**: 使用均匀的概率从 P_S 中选择 $m\%$ 的成员。对选出来的每一个成员, 在它的表示中随机选择一个位取反;
 4. **更新**: $P \leftarrow P_S$;
 5. **评估**: 对于 P 中每一个假设 h 计算 $\text{Fitness}(h)$;
 - }
- 从 P 中返回适应度最高的假设

Fitness: 适应度评分函数

fitness_thr: 指定终止判据的阈值

p: 群体中包含的假设数量

r: 每步通过交叉取代群体成员比例

m: 变异率

二、遗传算法原型

- 遗传算法有以下参数：
 - 交叉率Crossover rate
 - 变异率Mutation rate
 - 群体大小Population size
 - 编码Encoding
 - 交叉变异类型Crossover and mutation type
 - 选择Selection
- 没有一种通用理论能解决各种问题调整参数
 - 可能需要针对特定问题尝试自己的遗传算法
- 遗传算法的两个核心问题
 - 表示假设：二值、实数
 - 遗传算子：交叉、变异、选择

二、遗传算法 --表示假设

- 假设经常用二进制位串表示
 - 便于用变异和交叉遗传算子来操作
- 例子: 编码if-then规则
 - If wind=strong, then playTennis =yes
 - 属性编码
 - Wind: Strong, weak
 - 长为2的位串: 10
 - 编码属性不被规则约束的前件, “不关心属性值”
Outlook = 111 (sunny, overcast, rain) “任意值”
 - 规则编码
 - 连接相应前件和后件的位串连接
 - outlook wind PlayTennis
 - 111 10 10

二、遗传算法 --表示假设

- 规则集的表达

- 表示规则的位串对假设空间中的每个属性有一个子串
- 得到固定长度的规则位串表示
- 特定位置的子串描述对特定属性的约束
- 以上单个规则的表示方法，可以简单地进行连接形成表示规则集

- 每个句法合法的位串表示一个有意义的假设

- 111 10 11 也可以表示一个规则
- 有什么问题吗？
- 采用不同的编码方式 或 改变遗传算子 或 赋值很低适应度

二、遗传算法 --遗传算子

交叉操作

交叉操作是指按照某种方式对选择的父代个体的染色体的部分基因进行交叉重组，从而形成新的个体。

交叉重组是自然界中生物遗传进化的一个主要环节，也是遗传算法中产生新的个体最重要的方法之一。根据个体编码方法的不同，遗传算法中的交叉操作可分为**二进制交叉**和**实值交叉**两种类型。

二进制交叉是指二进制编码情况下所采用的交叉操作，它主要包括**单点交叉**、**两点交叉**和**均匀交叉**等方法。

二、遗传算法

遗传算子

● 交叉算子

- 从双亲串中通过复制选定位产生两个新的后代
- 交叉掩码:

✓ 单点交叉

A: 1011011100 1111111100 → A': 1011011111
B: 0001110011 → B': 0001110000

✓ 两点交叉

A: 1011011100 0001111100 → A': 1011011100
B: 0001110011 → B': 0001110011

✓ 均匀交叉

A: 1011001100 0001101100 → A': 0001011111
B: 0001110011 → B': 1011100000

二、遗传算法 ——遗传算子

变异操作

变异是指对选中个体的染色体中的某些基因进行变动，以形成新的个体。变异也是生物遗传和自然演化中的一种基本现象，它可增强种群的多样性。遗传算法中的变异操作增加了算法的局部随机搜索能力，从而可以维持种群的多样性。

二进制变异：该变异方法是先随机地产生一个变异位，然后将该变异位置上的基因值由“0”变为“1”，或由“1”变为“0”，产生一个新的个体。

例如：设变异前的个体为 $A=0\ 0\ 1\ 1\ 0\ 1$ ，若随机产生的变异位置是2，则该个体的第2位由“0”变为“1”。变异后的新的个体是 $A'=0\ 1\ 1\ 1\ 0\ 1$ 。

二、遗传算法

遗传算子 - 变异

- 变异算子：

- 对单亲的位串产生随机的小变化
- 方法是选取一个位，然后取反
- 变异经常是应用了交叉之后进行的

- 其他算子

- 特别是一些针对系统中特定假设表示的算子
- 描述机器人控制规则集的系统

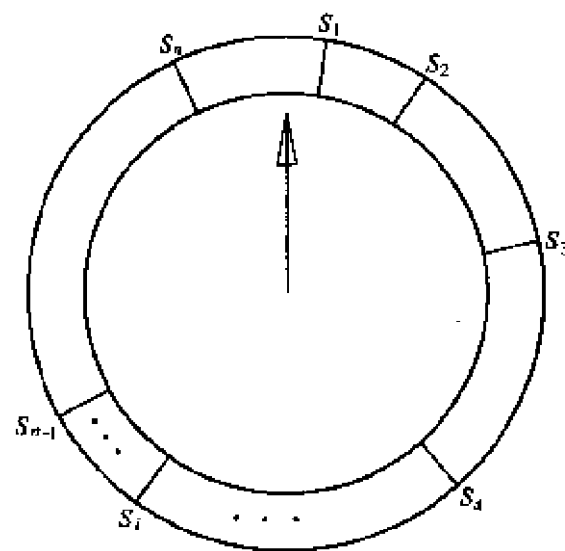
- 学习规则集的系统

- 直接使用了多种直接泛化和特化规则的算子

二、遗传算法

适应度函数和假设选择

- **适应度函数**定义了候选假设的排序标准
 - 以概率方法选择下一代群体的准则
- 选择某假设概率的一个可能的方法
 - 适应度比例选择 (轮盘赌选择)
 - 锦标赛选择
 - 随机选取两个假设
 - 事先定义 $p, (1 - p) \rightarrow h_{high}, h_{low}$
 - 相比之下, 更加多样化的群体
 - 排序选择
 - 按适应度排序
 - 某假设概率与其排序位置成比例



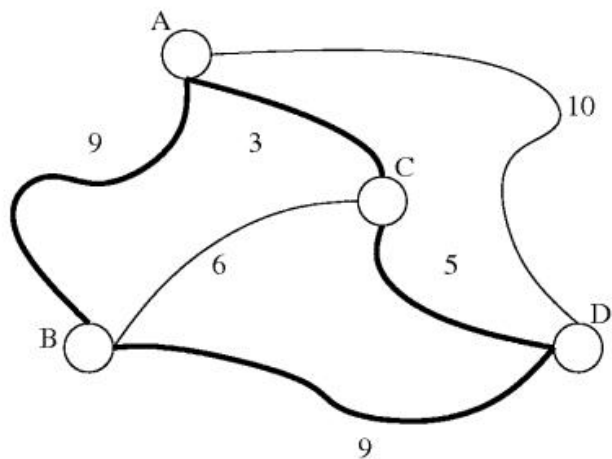
三、遗传算法举例

旅行商问题

- 设有 n 个互相可直达的城市，有 n 个城市和距离矩阵，其中， d_{ij} 表示城市 i 到城市 j 的距离， $i, j=1, 2 \dots n$ ，则问题是要找出遍访每个城市恰好一次的一条回路并使其路径长度为最短。
- 这个问题已被证明为NP-完全问题
 - 如果想找到真正的唯一最优的解复杂度是 $O(N!)$ 的
- 解决方案：
 - 启发式 heuristics
 - 割平面法 cutting-plane methods
 - 分支定界 branch-and-cut
- 如何用遗传算法解决？

三、遗传算法举例

- 编码



1st city 2nd city last city
on tour on tour on tour

City #

--	--	--	--

例如，我们有6个城市，一个旅游方案为3-1-2-4-5-6

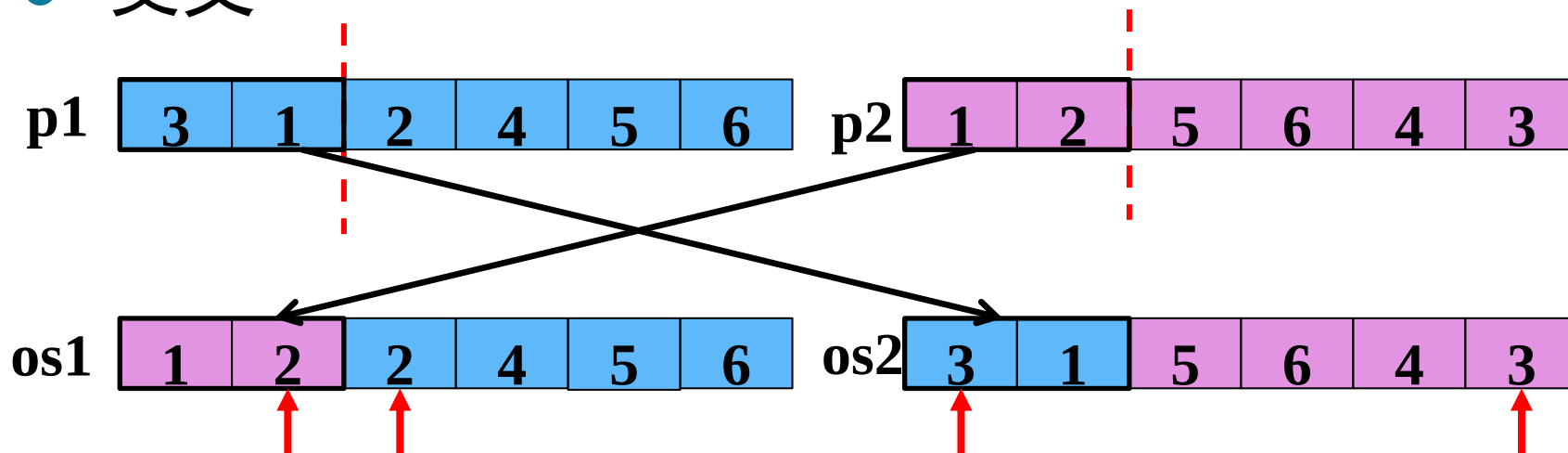
3	1	2	4	5	6
---	---	---	---	---	---

三、遗传算法举例

- 适应度评估

- 给定一条染色体，只需计算总距离即可
- 距离越短，染色体的适应性越高

- 交叉

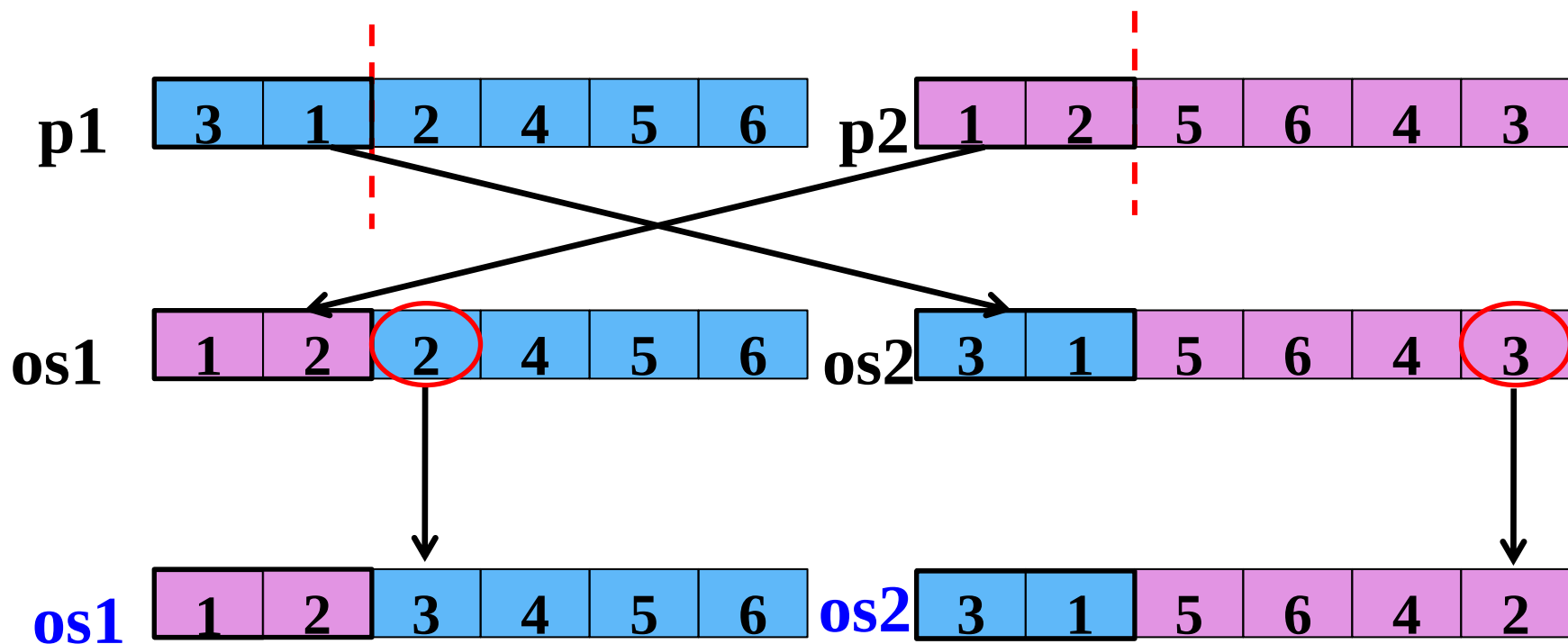


- 你发现了什么问题吗？

三、遗传算法举例

如何解决这个问题？

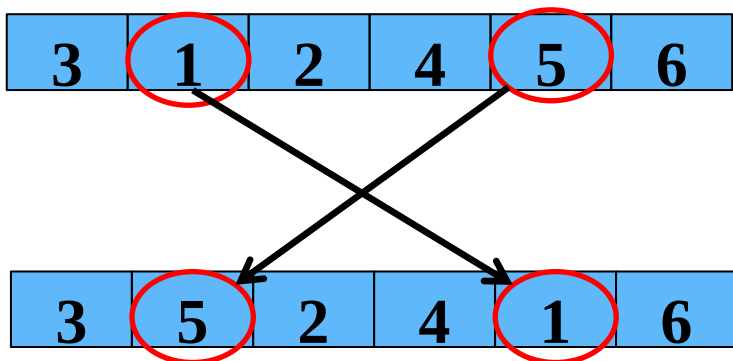
- 修复算子



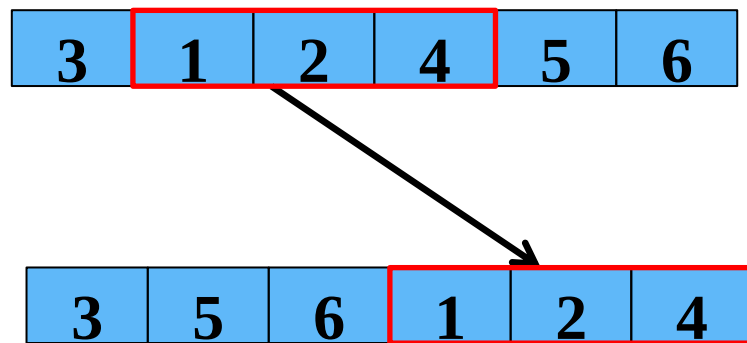
三、遗传算法举例

- 变异算子

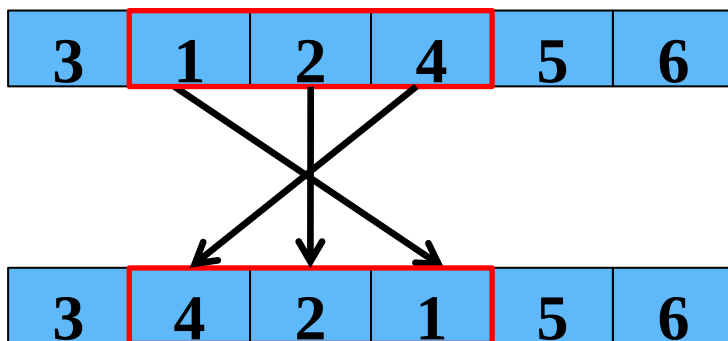
- 交叉变异



- 替换变异



- 简单倒置变异



三、遗传算法练习

练习三、在利用遗传算法解决旅行商问题时，若对下列一个染色体进行变异操作，请写出变异结果。
(假设变异点基因位置为3位和5位)。

A: 2 8 4 10 5 1 7 3 6 9

四、假设空间搜索

- 遗传算法采用随机柱状搜索方法来寻找最大适应度假设
 - 梯度下降搜索：从一个 h 平滑地移动到新的 h
 - 遗传算法：移动可能非常突然，用完全不同假设替代双亲
 - ▣ 不太可能陷入相同的局部极小值
- 一个实际困难：拥挤问题
 - 适应度高的个体迅速繁殖，因此该个体和它非常相似的个体占据了群体的很大一部分
 - 减少群体多样性，减缓进一步进化

四、假设空间搜索

● 降低拥挤的策略

➤ 修改选择函数

- 使用锦标赛选择或排序选择

➤ 适应度共享

- 根据群体中与某个体相似的个体数量，减小该个体的适应度

➤ 对可重组生成后代的个体种类进行限制

- 只允许最相似的个体重组
- 按空间分布个体，并且仅允许相邻的个体重组

总结

- 遗传算法的优点

1. 算法不需要太多的先验知识或计算损失函数的梯度
2. 多目标函数优化
3. 可并行
4. 易处理庞大难理解的搜索空间
5. 广泛应用于优化问题求解

总结

- 遗传算法的缺点：

1. 假设的编码表示较难定义
2. 理想的适应度函数较难定义
3. 发生过早收敛
4. 选择诸如种群规模，变异率，交叉率，适应度函数之类的各种参数的问题
5. 无法使用梯度
6. 终止条件判断较困难
7. 需要大量的适应度计算

扩展：演化神经网络

演化神经网络概述

演化神经网络是基于演化计算和神经网络两大研究方向，将二者有机融合而产生的一种全新神经网络模型。

这种模型把演化计算的自适应机制与神经网络的学习机制有机的结合在一起，有效地克服了传统人工神经网络的很多缺点。

演化神经网络模型的一个主要特点就是它对动态环境的自适应性。这种自适应性过程通过演化的三个等级实现，即连接权值和阈值、网络结构和学习规则的演化。

Yao X, Evolving artificial neural networks, Proceedings of the IEEE, 87(9): 1423-1447, 1999. Citation: 4000+

扩展：演化神经网络

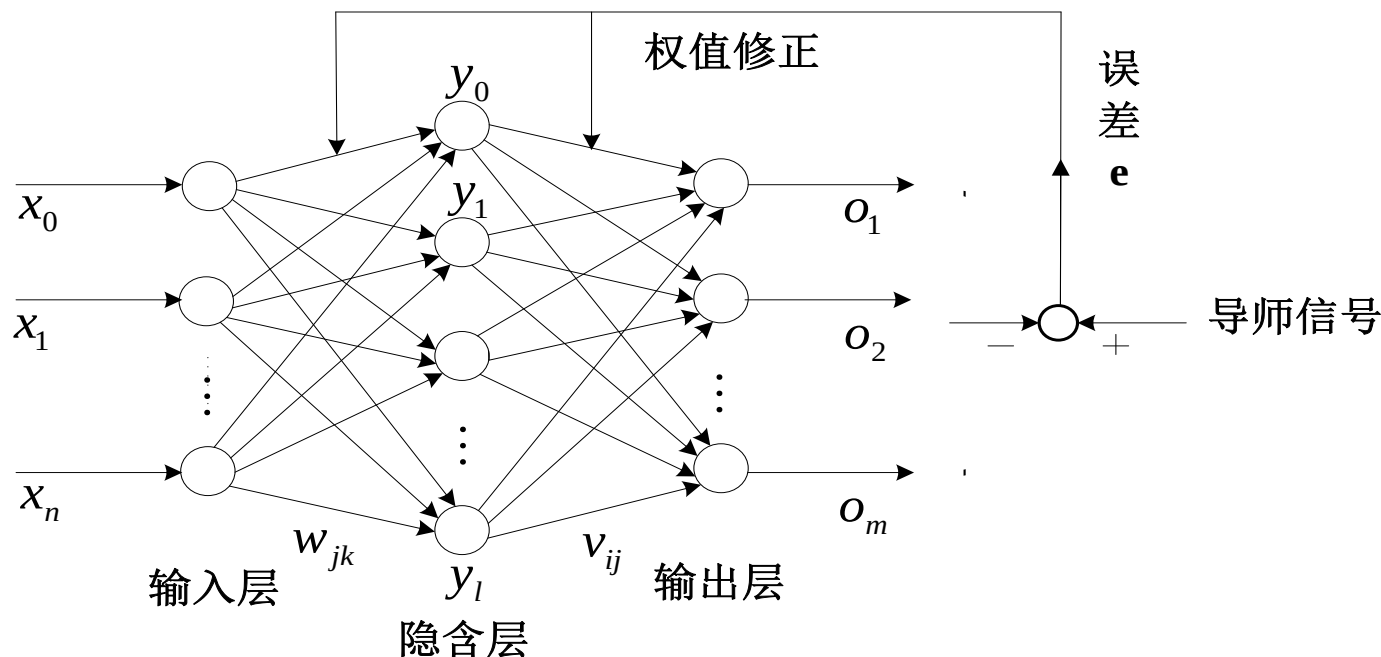
演化神经网络概述

根据演化神经网络实现的三个等级，演化神经网络模型有以下四种不同的类型：

- ◆ 初始权值和阈值演化（权值更新）
- ◆ 网络结构演化（直接/间接编码）
- ◆ 结构和权值阈值同时演化（EPNet系统）
- ◆ 学习规则演化（学习规则）

扩展：演化神经网络

初始权值阈值演化---以遗传算法优化BP神经网络的连接权值和阈值为例



- BP网络是一类多层的前馈神经网络，目前是人工神经网络中应用最广泛的算法之一
- 但是，存在一些缺陷，比如说学习收敛速度慢、不能保证收敛到全局极小点，网络结构不易确定。

扩展：演化神经网络

初始权值阈值演化

优化目标：用遗传算法优化BP神经网络的初始权值和阈值，使优化后的BP神经网络具有更好的预测精度。

算法的基本思路：

- (1) 对神经网络的初始权值和阈值进行**编码**
- (2) 然后，对种群所有个体进行解码，**生成**多个神经网络
- (3) 对每个神经网络进行BP**训练**，然后以均方根误差作为评价标准，对种群中所有个体进行**适应度评价**。
- (4) 进行选择、交叉、变异操作，产生**新的种群**
- (5) 判断是否达到停止条件，否则转到（2）运行。

扩展：演化神经网络

以某型拖拉机的齿轮箱为工程背景，介绍使用基于遗传算法的BP神经网络进行齿轮箱故障的诊断。

样本特征															齿轮状态
0.2286	0.1292	0.072	0.1592	0.1335	0.0733	0.1159	0.094	0.0522	0.1345	0.009	0.126	0.3619	0.069	0.1828	无故障
0.209	0.0947	0.1393	0.1387	0.2558	0.09	0.0771	0.0882	0.0393	0.143	0.0126	0.167	0.245	0.0508	0.1328	无故障
0.0442	0.088	0.1147	0.0563	0.3347	0.115	0.1453	0.0429	0.1818	0.0378	0.0092	0.2251	0.1516	0.0858	0.067	无故障
0.2603	0.1715	0.0702	0.2711	0.1491	0.133	0.0968	0.1911	0.2545	0.0871	0.006	0.1793	0.1002	0.0789	0.0909	齿根裂纹
0.369	0.2222	0.0562	0.5157	0.1872	0.1614	0.1425	0.1506	0.131	0.05	0.0078	0.0348	0.0451	0.0707	0.088	齿根裂纹
0.0359	0.1149	0.123	0.546	0.1977	0.1248	0.0624	0.0832	0.164	0.1002	0.0059	0.1503	0.1837	0.1295	0.07	齿根裂纹
0.1759	0.2347	0.1829	0.1811	0.2922	0.0655	0.0774	0.0227 3	0.2056	0.0925	0.0078	0.1852	0.3501	0.168	0.2668	断齿
0.0724	0.1909	0.134	0.2409	0.2842	0.045	0.0824	0.1064	0.1909	0.1586	0.0116	0.1698	0.3644	0.2718	0.2494	断齿
0.2634	0.2258	0.1165	0.1154	0.1074	0.0657	0.061	0.2623	0.2588	0.1155	0.005	0.0978	0.1511	0.2273	0.322	断齿

一共9个训练样本，每个样本为15维特征向量，所以神经网络输入层15个神经元。齿轮有3种故障模式，分别为无故障，齿根裂纹断齿，所以输出层为3个神经元。

作业题

在利用遗传算法解决旅行商问题时，若采用基于位置的交叉方式对下列两个染色体进行交叉，请写出交叉结果。

A: 2 8 4 10 5 1 7 3 6 9

B: 5 6 7 1 10 2 8 3 9 4